



Linux - Scripting

Formateur : Sébastien GIRAUD

OPEN YOUR
FUTURE

FORMATION ECOLE



OPEN
SOURCE
SCHOOL

OPEN YOUR
PERSPECTIVES

FORMATION CONTINUE

Bienvenue !

- Acquérir une connaissance des divers aspects du shell
- Connaître tous les mécanismes de base du shell
- Créer des scripts transportables d'un Linux à l'autre
- Une passe sur des mécanismes avancés du shell vous permettra de bien maîtriser l'outil
- Connaître toutes les astuces du shell ainsi que des commandes d'administration basiques du système Linux

Plan de la formation 1/2



Introduction

- Historique
- Présentation
- Fichiers de configuration
- Créer un script shell
- Exécution d'un script

Mécanismes de base

- Affichage et lecture
- Commentaires
- Les variables
- L'environnement
- Les quotes
- Les arguments
- Codes de retour

Plan de la formation 2/2



Construction de shell scripts portables

- If et case
- Les comparaisons
- For et while
- Les fonctions
- L'import de fichiers

Mécanismes complémentaires

- Les redirections
- Opérations mathématiques
- Meta characters
- ANSI-C
- Getopts
- Les tableaux
- Le select
- Les signaux

Commandes et rappels de base 1/3

Commandes de base, toutes les commandes ont une signification

- `whoami` Identité
- `date` Date et heure de la machine
- `pwd` Chemin courant (print working directory)
- `who` Liste des utilisateurs connectés
- `cal` Calendrier
- `uname` Informations sur le système
- `passwd` Changement du mot de passe
- `man` Documentation standard (manuels)

Gestion de fichiers

- `ls` Afficher les fichiers du dossier (list)
- `cd` Changement de dossier (change directory)
- `cp` Copie de fichier (copy)
- `mv` Déplacement de fichiers (move)

Commandes et rappels de base 2/3



Gestion de fichiers (suite)

- `rm` Suppression de fichiers et dossiers (remove)
- `mkdir` Création de dossiers (make directory)
- `cat` Affichage de fichiers (concatenate and print...)

Autres commandes

- `su` Changement d'identité (switch user)
- `sudo` Lancer une commande en tant qu'un autre utilisateur

Utilisation du clavier

- Touches fléchées du clavier (historique, etc.)
- `Ctrl+a` Aller au début de la ligne
- `Ctrl+e` Aller à la fin de la ligne
- `Ctrl+c` Supprimer la ligne, terminer le programme en cours
- `Ctrl+d` Quitter le shell (exit)

Commandes et rappels de base 3/3

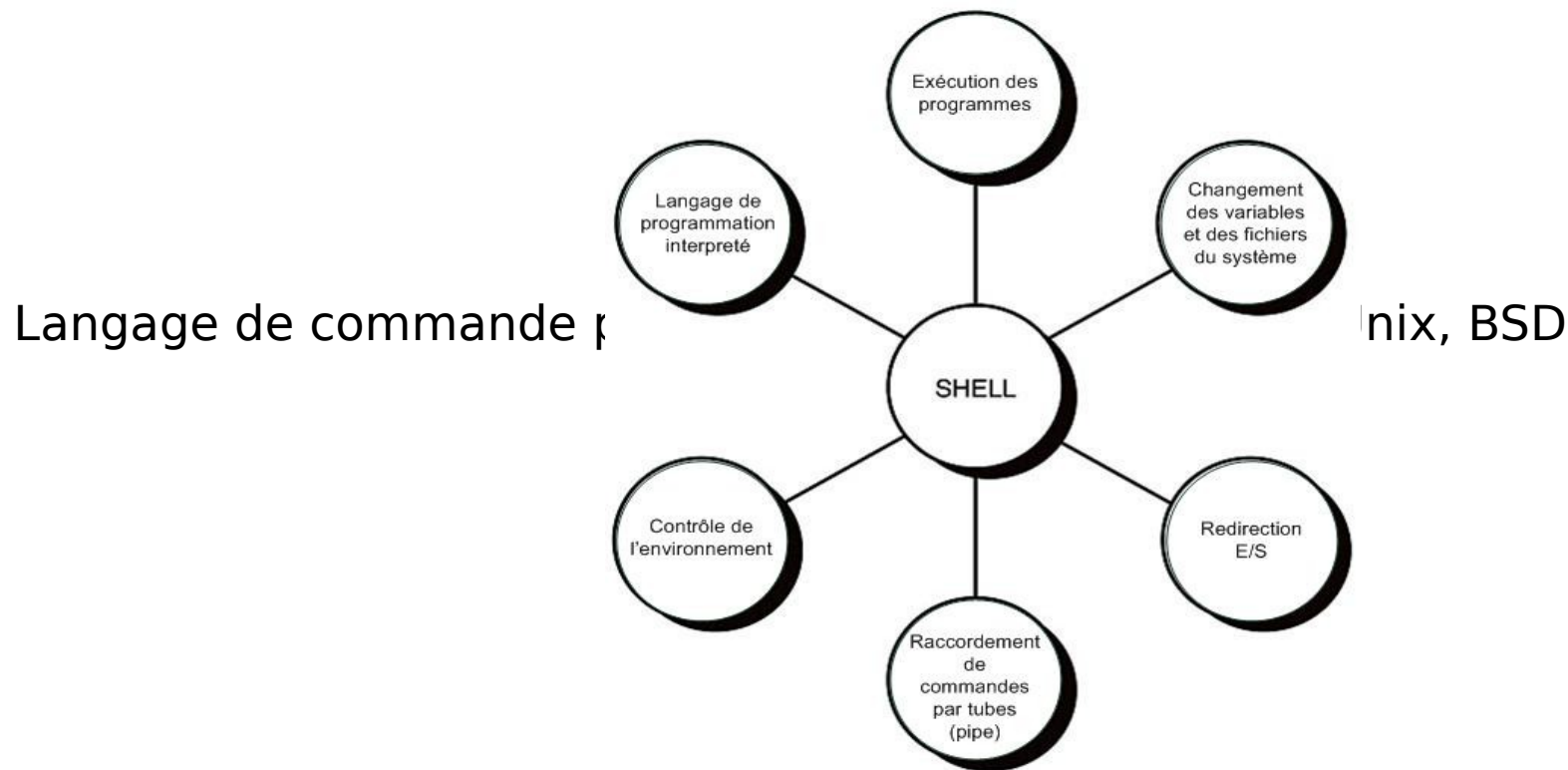
Utilisation du clavier (suite)

- **Tabulation** Complétion automatique (« auto completion »)
 - **Ctrl+r** Rechercher une commande dans l'historique
 - **Ctrl+k** Couper tout après le curseur
 - **Ctrl+u** Coupe tout avant le curseur (efface la saisie)
 - **Ctrl+y** Coller ce qui a été coupé
 - **Ctrl+Flèche** Déplacement mot par mot
 - **Alt+Backspace** Suppression de mots avant le curseur
- Attention à **Ctrl+s** : gèle l'affichage dans le terminal (**Ctrl+q** pour dégeler)

Introduction – Historique 1/2

1977 - Le Bourne Shell **sh** est le premier interpréteur de commandes inventé par Steve Bourne

Interface Homme-Machine, les commandes sont exécutées par le cœur du système (« kernel ») et permettent des facilités courantes telles que :



Introduction – Historique 2/2



1978 - La branche BSD développe parallèlement le C-shell **cs**h pour ses propres évolutions techniques

- Installé par défaut sur FreeBSD

1983 - David Korn ajoute des fonctions supplémentaires (rappels de commandes) dans son Korn Shell **ks**h

- Installé par défaut sur Solaris

1989 - La Free Software Foundation propose un nouveau shell en réaction aux shells propriétaires : Bourne-Again Shell **ba**sh

- Installé par défaut sur la plupart des distributions Linux

1992 - L'IEEE élabore le Posix Shell ou normalisation de **s**h

Introduction – Présentation 1/2

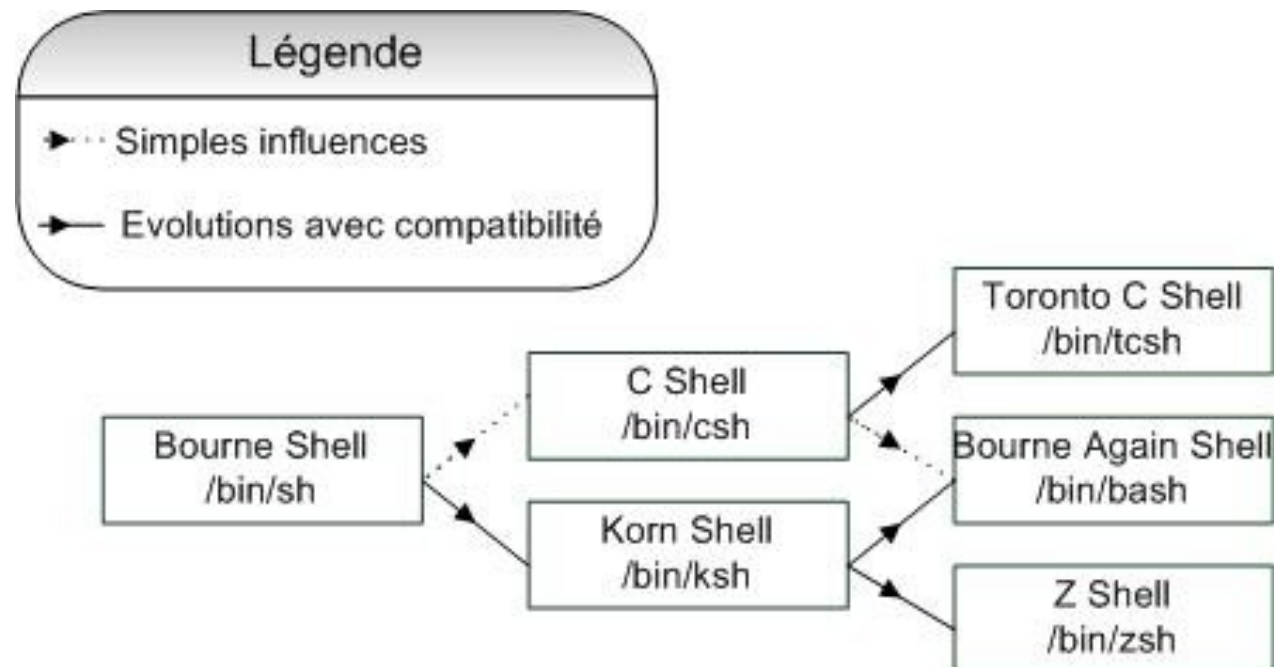
Bash est le Bourne-Again Shell (« Born Again ») du projet GNU

Licence GPLv3

Compatibilité :

Peut être compatible POSIX avec l'option `--posix`

Pas de typage des variables en Bash !



Introduction – Présentation 2/2

prompt

commande

Résultat de la commande

```
root@srv-formation:~# ls -l /
total 68
lrwxrwxrwx.    1 root root    7 30 août   2018 bin -> usr/bin
dr-xr-xr-x.    6 root root 3072 21 oct.   2018 boot
drwxr-xr-x.   19 root root 3060 28 août   12:15 dev
drwxr-xr-x.  102 root root 12288 15 août   06:55 etc
drwxr-xr-x.   20 root root 4096 22 juil.  14:20 home
lrwxrwxrwx.    1 root root    7 30 août   2018 lib -> usr/lib
lrwxrwxrwx.    1 root root    9 30 août   2018 lib64 -> usr/lib64
drwx-----.    2 root root 16384 11 déc.   2016 lost+found
drwxr-xr-x.    2 root root 4096 11 avril  2018 media
drwxr-xr-x.    2 root root 4096 11 avril  2018 mnt
drwxr-xr-x.    2 root root 4096 11 avril  2018 opt
dr-xr-xr-x.  146 root root    0 28 août   00:45 proc
dr-xr-x---.    9 root root 4096 16 sept.  14:28 root
drwxr-xr-x.   34 root root 1120  1 sept.  00:00 run
lrwxrwxrwx.    1 root root    8 30 août   2018/sbin -> usr/sbin
drwxr-xr-x.    2 root root 4096 11 avril  2018 srv
dr-xr-xr-x.   13 root root    0 28 août   00:46 sys
drwxrwxrwt.   10 root root 4096  4 nov.   17:35 tmp
drwxr-xr-x.   13 root root 4096 30 août   2018 usr
drwxr-xr-x.   22 root root 4096 20 oct.   2018 var
```

Introduction – Fichiers de configuration 1/2



Rappel : Linux est un système orienté multi-utilisateurs

Fichiers de configuration communs à tous les utilisateurs :

- `/etc/profile` : exécuté au démarrage d'un shell interactif avec connexion
- `/etc/bashrc` ou `/etc/bash.bashrc` exécuté au démarrage d'un shell interactif
- ATTENTION : Il peut être dangereux de modifier ces fichiers

Fichiers de configuration propres à chaque utilisateur :

- `~/.bashrc` et `~/.bash_profile` : Ces fichiers sont des versions par utilisateur exécutées après leur équivalent dans `/etc/bashrc/{profile,bash.bashrc}`
- `~/.bash_logout` : Commandes à exécuter avant de déconnecter l'utilisateur
- `~/.bash_history` : Fichier contenant l'historique des commandes exécutées dans le shell et qui peuvent être rappelées

Introduction – Fichiers de configuration 2/2



Exemple d'un ~/.bashrc :

```
# Import du fichier de configuration de base
. /etc/bash.bashrc

# PS1 – Variable permettant de customiser le prompt
if [[ ${EUID} == 0 ]] ; then
    PS1='\[\033[01;31m\]\u\[\033[00m\]@\h:\w# '
else
    PS1='\[\033[01;31m\]\u\[\033[00m\]@\h:\w\$ '
fi

# exports – Ils permettent d'ajouter des variables a l'environnement
# de l'utilisateur (les processus enfants y auront accès)
export PS1="$PS1"
export BRIAN="KITCHEN"

# alias – autorisent, entre autre, les utilisateurs a donner des
# petits noms a leurs commandes favorites
alias cp='cp -i'
alias rm='rm -i'
alias mv='mv -i'
alias s='sudo'

# Il est possible de lancer des commandes au chargement du fichier
echo "Welcome !"
```

Introduction – Exercice



Copier votre `~/ .bashrc` en `~/ .bashrc.bak`

Éditez avec votre éditeur favori le `~/ .bashrc` et modifiez le de la façon suivante :

- A partir de l'exemple montré en cours, modifiez le `PS1` pour qu'en root, le nom s'affiche en rouge et pour le reste, la couleur de votre choix
 - Astuce : *Black 01;30, Blue 01;34, Green 01;32, Cyan 01;36, Red 01;31, Purple 01;35, Brown 01;33*
- Exportez les variables `PS1`, `PAGER` et `EDITOR` telles que :
 - `PS1` prenne en compte vos modifications précédentes
 - `PAGER` soit `less`, `more` ou `most` (vérifiez leur existence avec `which`)
 - `EDITOR` soit votre éditeur favori (`emacs`, `vi/vim`, `nano`, vérifiez leur existence avec `which`)
- Ajoutez les alias suivants :
 - `ll`: commande `ls` avec affichage des fichiers cachés, au format long, et « human readable »
 - `rm`: lance la commande `rm` avec demande de confirmation avant effacement
 - `df`: lance la commande `df` au format « human readable »
 - Astuces : `RTFM !`, `man ls`, `man rm`, `man df`

Tester avec la commande : `$ . ~/ .bashrc`

Introduction – Historique

L'historique des commandes permet de retrouver ce qui a été exécuté.

```
segir@srv-formation:~> history
```

```
 1  2015/10/29 - 15:25:59 rsync -av 5.44.23.154:/tmp/Pre* .
 2  2014/07/30 - 16:00:18 ls -ltr
 3  2014/07/30 - 16:04:17 sssh sd-19459.dedibox.fr
 4  2014/07/30 - 15:56:01 cd /tmp
 5  2014/07/30 - 15:56:03 ls -altr
 6  2014/07/30 - 16:06:53 pwgen
 7  2014/07/30 - 16:08:23 ftp ftp.smile.fr
 8  2014/07/30 - 16:08:59 ssh puma
 9  2014/07/30 - 15:51:19 vi /home/hosting/clients/clientA/contact.txt
10  2014/07/30 - 15:51:59 ls
```

```
segir@srv-formation:~>
```

Variables de contrôle de l'historique dans `~/.bashrc`

```
HISTSIZE=100000
```

```
HISTIGNORE=ls:free:du:jobs
```

```
HISTCONTROL=ignoredups
```

```
HISTTIMEFORMAT="%Y/%m/%d - %H:%M:%S "
```


Quand écrire un script ?



Les shell scripts servent à créer des enchaînements de commandes.

L'objectif étant d'automatiser des suites de commandes qu'on lance répétitivement.

Dès qu'on a besoin de traiter des chaînes de caractères, de faire de l'algorithmique, il faut passer à un langage évolué, Python, par exemple.

Car sinon, à force d'enrichir ses shell scripts on obtient du code difficile à maintenir, car il n'y a pas de correcteur syntaxique ni de débogueur.

Perl peut être utilisé pour des besoins personnels, c'est à dire se simplifier la vie avec des scripts à soi. Mais dès qu'on souhaite partager des scripts, il faut utiliser soit du Shell, soit du Python.

Introduction – Créer un script bash 1/2

Commencer par vérifier où se trouve l'exécutable `bash` avec la commande :

- `$ which bash`
- `/bin/bash`

Ouvrir avec son éditeur favori un fichier `~/test.sh`

A la première ligne, rajout du « Sha-Bang » tel que :

- `#!/bin/bash`
- Note : Il n'est pas obligatoire d'ajouter cette directive mais il est vivement recommandé de le faire pour indiquer au lecteur quel est le programme valide pour exécuter la commande

Rajouter le code souhaité

- Exemple : `echo 'Hello World !'`

Introduction – Créer un script bash 2/2



Terminer le fichier avec la directive :

- `exit 0`
- Le code 0 permet d'indiquer au shell que votre programme s'est bien terminé, utiliser un autre chiffre pour lui dire le contraire

Sauvegarder et quitter le fichier

Ajouter les droits d'exécution au fichier :

- `$ chmod a+x ~/test.sh`

Exécuter le programme de cette manière :

- `$ ~/test.sh`

Introduction – Exécution d'un script 1/4

Pour des raisons pratiques et également de sécurité, un subshell (sous-shell) est créé et exécute le script, puis renvoie le code de retour

- c'est le cas le plus courant
- c'est un fork ! L'environnement (variables globales) est repris
- exemple : `$ /chemin/vers/mon/script`
- le script doit être exécutable (`chmod +x`)

Le script peut aussi être exécuté dans le shell courant, sans fork

- plutôt dangereux, permet de modifier l'environnement
- utilisé par exemple pour `~/.bashrc`
- exemple : `$ source /chemin/vers/monscript`
- syntaxe alternative : `$ . /chemin/vers/monscript`
- on dit alors qu'on « source » le script

Introduction – Exécution d'un script 2/4



On exécute un script par son chemin complet.

On modifie son environnement en sourçant un fichier qui contient les déclarations de variables

A noter qu'en administration système, on ne modifie pas le `$PATH`, afin de ne pas surprendre les admins (sauf cas particuliers)

Introduction – Exécution d'un script 3/4



Un shell parent qui forke un sous-shell attend que le processus enfant lui renvoie un code retour pour rendre la main :

- problématique si la tâche est longue (exemple : dump de base de données...)

On exécute la tâche directement en arrière-plan, en suffixant par le caractère `&` :
`$ matache&`

On peut surveiller l'exécution via la commande `jobs`

Les commandes `fg` (foreground) et `bg` (background) permettent respectivement de ramener un script lancé en arrière-plan dans l'exécution courante, ou de l'y renvoyer

Un processus peut être suspendu en pleine exécution (`Ctrl+Z`) puis passé en arrière-plan grâce à `bg`

On peut évidemment lancer autant de tâches en parallèle que possible par ce biais

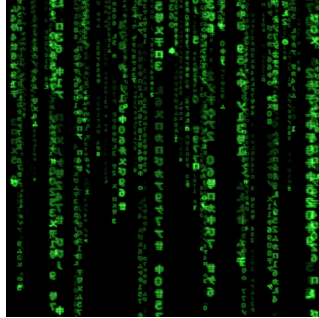
Introduction – Exécution d'un script 4/4

```
segir@svr-formation$ sleep 2000&
[1] 1077
segir@svr-formation$ jobs
[1]+  Running                  sleep 2000 &
segir@svr-formation$ fg
sleep 2000
^Z
[1]+  Stopped                  sleep 2000
segir@svr-formation$ jobs
[1]+  Stopped                  sleep 2000

segir@svr-formation$ bg
[1]+ sleep 2000 &

segir@svr-formation$ jobs
[1]+  Running                  sleep 2000 &
```

A wild admin appears



Quand des scripts sont très longs et que vous ne pouvez pas maintenir de connexions (ex : *ssh*), la commande *nohup* suivie de votre script permet de lancer votre programme sur un shell détaché du vôtre.

Les commandes *screen* et *tmux* permet la même astuce avec en plus la possibilité de travailler en équipe en partageant un même shell visible à plusieurs.

Mécanismes de base – Affichage et lecture



La commande permettant l'affichage :

```
$ echo 'Bonjour tout le monde'
```

```
Bonjour tout le monde
```

- Option pratique : `-n`, évite de faire un saut de ligne après l'affichage
- Pour les autres options : RTFM !

La commande permettant de lire les demandes d'utilisateur :

```
$ echo -n 'Quel age as-tu ? ' && read
```

```
Quel age as-tu ? 18 ans
```

```
$ echo $REPLY
```

```
18 ans
```

- `$REPLY` est une variable permettant de recueillir tout ce qu'une personne écrit pendant un `read`
- Vous pouvez utiliser d'autres variables pour récupérer les mots lus :

```
$ echo -n 'Ecris 2 mots : ' && read mot1 mot2
```

```
Ecris 2 mots : Hello world
```

```
$ echo $mot1
```

```
Hello
```

Mécanismes de base – Commentaires



Pour écrire des commentaires dans un script, on fait précéder le début de la ligne par # comme ceci :

```
#!/bin/bash
```

```
# Un bon commentaire explique ce que fait le programme  
# Il ne s'agit pas traduire en français ce que font les instructions  
# On suppose que le lecteur connaît le langage.  
# On ne paraphrase pas le code !!!
```

```
echo "parce que sinon vos programmes sont illisibles"
```

```
exit 0 # vos commentaires peuvent aussi s'écrire en fin de ligne
```

Ne vous privez pas non plus d'espacer vos lignes dans le script.

L'objectif de l'écriture consiste à faciliter la lecture. Un code "abscons" ira un jour à la poubelle, et souvent plus vite que prévu...

Des rappels fréquents seront faits sur les exemples de ce cours.

Mécanismes de base – Les variables 1/4

Première Partie : Pas de typage fort
Une variable peut contenir un mot :

```
$ MA_VARIABLE_0=coucou
```

Ou une phrase si elle est entourée de " ou ' :

```
$ VAR="TOUT le monde"
```

Ou encore un nombre :

```
$ UNE_AUTRE_VAR=42
```

Attention : pas d'espaces autour du "=" dans les déclarations de variables
Les variables sont écrites en majuscule de préférence pour les différencier du reste

Les caractères numériques sont autorisés ainsi que underscore « _ » mais pas le tiret « - »

Mécanismes de base – Les variables 2/4

Deuxième partie : les manipulations

Le résultat d'une commande peut-être enregistré dans une variable :

```
$ LS=$(ls -larth /)
```

C'est aussi possible grâce aux backquotes ` (alt gr+7)(back sticks) :

```
$ LS=`ls -larth /`
```

Vous pouvez aussi afficher une variable :

```
$ echo "$LS"
```

Récupérer dans des variables le résultat d'une lecture (Attention : chaque variable stocke un mot) :

```
$ read MOT1 MOT2
```

Afficher le contenu des variables après lecture :

```
$ echo "$MOT1" "$MOT2"
```

Mécanismes de base – Les variables 3/4

Attentions aux espaces : séparateurs de variables

D'une manière générale, on évite les « espaces »

Déclarons 2 variables :

```
$ VAR1=Le  
$ VAR2=/etc
```

On peut concaténer des variables avec des lettres/chiffres :

```
$ FUSION="$VAR1 chemin $VAR2 contient "  
$ echo "$FUSION"
```

On peut aussi concaténer des commandes :

```
$ FUSION="$FUSION `ls /etc`"  
$ echo "$FUSION"
```

Mécanismes de base – Les variables 4/4

Troisième partie : les calculs

Déclarons 2 variables :

```
$ NUM1=6
```

```
$ NUM2=7
```

`let` permet de calculer des expressions numériques :

```
$ let MULTI="$NUM1 * $NUM2" # ou aussi : let MULTI="6 * 7"
```

La syntaxe alternative d'expression fonctionne aussi :

```
$ ((MULTI=$NUM1 * $NUM2)) # ou aussi : MULTI=$(( $NUM1 * $NUM2 ))
```

Il est également possible de forcer le type entier pour une variable :

```
$ typeset -i NUM1
```

```
$ NUM1=0
```

```
$ NUM1=$NUM1+1
```

```
$ echo "$NUM1"
```

Mécanismes de base – Exercice 1

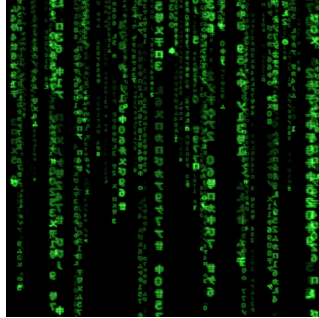
Écrire un script qui affiche :

- Le contenu du fichier `/etc/passwd`
- La première colonne du fichier précédent (les colonnes sont séparés par le caractère ' : ')
- Le nombre de lignes du fichier `/etc/group`
- L'affichage du fichier `/etc/hosts` en remplaçant les mots « localhost » par le nom de votre machine
- Votre âge, que vous vous serez demandé par l'intermédiaire du script, multiplié par 6

Consignes :

- Seule la commande `echo` a le droit de faire des affichages
- Vous n'avez le droit qu'à une seule commande `echo` par consigne
- Vous n'avez le droit d'utiliser qu'une seule variable par consigne
- Vous devrez commenter et aérer votre code ainsi que l'affichage des résultats (une autre commande `echo` peut être utilisée pour cet usage)

Conseil de l'admin



Recherche dans un **man** :

- Appuyer sur /le_truc_cherche
- Appuyer sur Entrée puis **n** jusqu'à arriver où on veut

Compter des caractères, des lignes, etc : **wc**

Afficher un fichier : **cat**

Découper par colonnes : **cut**

Remplacer des caractères : **sed** (chercher dans le **man s/**)

Autres commandes utiles :

tail, head, tr, sort, find, grep

Mécanismes de base – L'environnement 1/4

Le shell est exécuté dans un processus : il possède donc un environnement d'exécution

L'environnement est constitué de variables (et leurs valeurs), de deux types :

- Variables propres au processus
- Variables héritées (transmises aux processus enfants)

Comme dans tout programme, la modification de ces variables a un impact direct sur le fonctionnement

Exercice : Listing des variables d'environnement de votre shell

- Exécutez `/bin/bash`
- Lancez les commandes `env` et `set`, décrivez le résultat

Mécanismes de base – L'environnement 2/4

```
segir@svr-formation:~$ env
LC_PAPER=fr_FR.UTF-8
LC_ADDRESS=fr_FR.UTF-8
HOSTNAME=svr-formation
SELINUX_ROLE_REQUESTED=
TERM=xterm
SHELL=/bin/bash
HISTSIZE=10000
SSH_CLIENT=10.2.1.5 53722 22
SELINUX_USE_CURRENT_RANGE=
SSH_TTY=/dev/pts/0
USER=segir
SSH_AUTH_SOCK=/tmp/ssh-PEtqprvXnD/agent.27619
PAGER=/usr/bin/less
MAIL=/var/spool/mail/segir
PATH=/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/segir/.local/bin:/home/segir/bin
PWD=/home/segir
EDITOR=vi
LANG=en_US.UTF-8
PS1=ovh#\[\e]0;\h:\w\a\]\[\e[0;32m]\u@\h:\w\[\e[$((${???:0}));37m]\$\[\e[0;0m\]
HISTIGNORE=ls
HISTCONTROL=ignoredups
HOME=/home/segir
LS_OPTIONS=--color=auto
LESS=FiX
LOGNAME=segir
SSH_CONNECTION=10.2.1.5 53722 10.2.1.2 22
HISTTIMEFORMAT=%Y/%m/%d - %H:%M:%S
LC_NAME=fr_FR.UTF-8
_=/usr/bin/env
```

Mécanismes de base – L'environnement 3/4

La plupart des variables d'environnement sont normalisées (IEEE Std 1003.1)

env affiche les variables globales, **set** affiche **toutes** les variables

Quelques exemples prédéfinis de variables globales (à connaître !) :

- **USER** : le login de l'utilisateur du Shell (processus)
- **TERM** : type d'émulateur de terminal utilisé
- **HOME** : le répertoire de démarrage de l'utilisateur
- **PWD** : le répertoire courant (« print working directory »)
- **PAGER** : programme utilisé pour la découpe en page de longs fichiers
(Exemple : pages de manuel)
- **SHELL** : l'exécutable utilisé par ce même shell
- **HOSTNAME** : le nom de la machine
- **EDITOR** : l'éditeur de texte utilisé par les commandes d'édition
- **PATH** : la liste des chemins où trouver les exécutables uniquement connus par leur nom
 - autrement, il faut appeler les commandes par leur chemin complet

Mécanismes de base – L'environnement 4/4

Il est possible de modifier une variable d'environnement, avec la commande export :

```
$ export A=3  
$ export BRIAN=kitchen  
$ export PWD=/home
```

Sans la commande export, les variables sont « locales » et ne sont pas transmises à d'éventuels shells enfants

Pour afficher la valeur d'une seule variable d'environnement, on peut utiliser la commande « built-in » `echo` :

```
$ echo $HOME  
/home/wedjat
```

Mécanismes de base – Exercice 2

Dans un script `exo2-1.sh` :

- Afficher vos `USER`, `PATH` et `HOSTNAME`

Dans un script `exo2-2.sh` :

- Sauvegarder la variable globale `PATH` en une autre variable globale `PATH_OLD`
- Ajouter la ligne : `exo2-1.sh`
- Concaténer votre répertoire courant à `PATH` (avec `:` entre les 2)
- Afficher `PATH` et `PATH_OLD`
- Ajouter la ligne : `exo2-1.sh`

Les résultats de votre script doivent ressembler à ça (Commenter les résultats) :

```
$ ./exo2-2.sh
./exo2-2.sh: line 4: exo2-1.sh: command not found
/usr/bin:/bin:/usr/sbin:/sbin:/home/formation/tp #$PATH
/usr/bin:/bin:/usr/sbin:/sbin #$PATH_OLD
formation
most
c7-formation
```

Mécanismes de base – Les quotes 1/3



La liste des principaux « meta characters » :

- `$`, utilisé devant un mot, permet de récupérer le contenu d'une variable
- `*`, pour « tous », 0 ou plus de caractères. Exemple : `ls /etc/*.conf` ou `find /etc -name "*.conf"`
- `?`, pour « tous », 0 ou 1 caractère
- `#`, tout ce qui est derrière lui est un commentaire
- ``cmd`` : le « backquote » permet de récupérer la sortie d'une commande dans une variable. Exemple : `DNS=`grep nameserver /etc/resolv.conf | tail -1``
- `;` Permet d'enchaîner les commandes sur une seule ligne. Exemple : `cd ~
; ls`
- `&`, permet d'envoyer une commande en arrière plan "background"
- `\`, Annule l'interprétation des autres meta characters y compris lui même. Entraîne l'interprétation de nouveaux meta characters. Exemples :

```
$ VAR1=`cat /etc/passwd`
```

```
$ echo -e "Affichage de \"VAR1\" et \"\${VAR1}\":\\nVAR1\\n\${VAR1}"
```

```
$ echo -e Une\ phrase\ sans\ espaces\ et\ sans\ \"
```

Mécanismes de base – Les quotes 2/3

Les simple quotes (') suppriment l'interprétation de tous les meta characters.

Des doubles quotes (") peuvent être insérés entre des simples quotes.

Des simples quotes ne peuvent être insérés entre des simples quotes y compris avec le caractère \

```
$ # Declaration d'une variable
```

```
$ BASH_VAR="$TERM"
```

```
$ # affichage de la variable BASH_VAR
```

```
$ echo $BASH_VAR
```

```
xterm
```

```
$ # Interpretation des meta characters annulee avec '
```

```
$ echo '$BASH_VAR "$BASH_VAR"'
```

```
$BASH_VAR "$BASH_VAR"
```

Mécanismes de base – Les quotes 3/3

Les double quotes (") suppriment l'interprétation de tous les meta characters sauf \$, \ et `.

Tous les autres sont lus de la même manière que les simples quotes.

On peut utiliser les simples quotes dans des doubles quotes. Les doubles quotes devront être « échappées » (escape) par \ pour être prises en compte.

```
$ BASH_VAR="Bash Script"
```

```
$ echo $BASH_VAR
```

```
Bash Script
```

```
$ echo "c'est $BASH_VAR et \"$BASH_VAR\" utilisant les backquotes:  
`date`"
```

```
c'est Bash Script et "Bash Script" utilisant les backquotes: Thu Oct  
4 01:54:32 CEST 2012
```


Mécanismes de base – Exercice 3

Créer les deux scripts suivants :

- `exo3-1.sh` : Afficher en une seule commande tous les fichiers se terminant par `.log` dans le dossier `/var/log` (ainsi que ses sous-dossiers) et qui contiennent les mots `error` (en majuscule et minuscule). Afficher également le numéro de ligne

Astuce : Utiliser les commandes `grep` et `find`

- `exo3-2.sh` : A l'aide d'une seule variable et d'un seul `echo`, faites que votre script affiche ceci :

```
$ ./exo3-2.sh
```

```
"VAR" vaut 'toto' et pas "$VAR"
```

Mécanismes de base – Les arguments

Un script Bash peut recevoir des arguments en entrée :

```
$ script.sh argument1 argument2 argument3
```

- `$1` : premier argument (`argument1`)
- `$2` : second argument, `$3` : troisième argument...
- `$0` : le nom du script lui-même (`script.sh`)
- `$#` : nombre d'arguments du script (3)
- `$*` : liste des arguments (`argument1 argument2 argument3`)

Mécanismes de base – Exercices 4 et 5

- Écrire un script `exo4-1.sh` qui affiche ceci :

```
$ ./exo4-1.sh coucou salut
```

```
Nombre d'arguments: 2
```

```
Tous les arguments: coucou salut
```

```
Nom du programme: ./exo4.sh
```

```
Argument 1: coucou
```

```
Argument 2: salut
```

- Écrire un script `exo4-2.sh` qui prend en paramètre un mot. Le script doit renvoyer une liste de tous les fichiers du répertoire `/var/log` contenant le mot recherché

Mécanismes de base – Codes de retour

Une commande renvoie toujours un code de retour pour rendre compte du déroulement du programme

Cela permet en général d'agir automatiquement selon le résultat

- 0 : OK (le plus souvent)
- 1 : NOK
- toute autre valeur : dépend du programme, mais généralement NOK

La variable pré-définie `$?` renvoie le code de retour de la dernière commande lancée

Rappel : `exit 0` en fin de script permet d'indiquer au shell que son déroulement s'est bien passé

Mécanismes de base – Codes de retour

Petit exemple :

```
$ ls /home
```

```
formation
```

```
$ echo $?
```

```
0
```

```
$ ls /etc/zzzz
```

```
ls: cannot access /etc/zzzz: No such file or directory
```

```
$ echo $?
```

```
2
```



OPEN
SOURCE
SCHOOL

3. Construction de shells scripts portables

Scripts portables – le if 1/3

La commande située après le mot-clé `if` détermine la condition

Si elle est vraie, le shell exécute les instructions derrière `then`

Sinon, le shell exécute les instructions derrière `else`

```
if [ $# -lt 1 ]          # Si le nombre d'arguments est inferieur a 1
then                    # Alors
    echo "Usage : $0 argument1"
    exit 1              # Exit en NOK
else                    # Sinon
    echo "$1"
fi                      # Fermeture du if
```

Attention à l'instruction `fi` à la fin du script.

Laisser des indentations (tab) derrière les `then` ou `else` pour plus de clarté

Scripts portables – le if 2/3

Les crochets `[ ]` sont équivalents à :

- la commande `test (/usr/bin/test)`
- la commande interne (built-in) `test`
- la commande `/usr/bin/[`

Bien faire attention aux espaces à l'intérieur des `[ ]`

On peut utiliser le mot clé `elif` pour imbriquer des conditions :

```
if [ $# -lt 1 ]
then
    echo "Usage : $0 argument1"
    exit 1
elif test "$1" = 'coucou' # Sinon si $1 contient le mot coucou
then
    echo "coucou a vous aussi"
else
    echo "$1"
fi
```


Scripts portables – le if 3/3

Vous pouvez imbriquer les `if` :

```
if [ $# -lt 1 -o $# -gt 2 ] # Si $# est inferieur a 1 OU superieur a 2
then
    echo "Usage : $0 argument1 [argument2]"
    exit 1
else
    if [ -z "$2" ] # verifie si $2 est vide (l'inverse etant -n)
    then
        echo "$0 marche mieux avec 2 arguments"
    else
        echo "$1" "$2"
    fi
fi
```

On peut également utiliser `-a` entre 2 tests, il signifie « ET »

Scripts portables – Comparaisons 1/4

Les chaînes de caractères (string) ne se comparent pas de la même manière que les nombres.

On peut ainsi tester des valeurs numériques comme ceci :

if test entier1	<code>-eq</code>	entier2	égal à
	<code>-ne</code>		non égal à
	<code>-gt</code>		supérieur à
	<code>-lt</code>		inférieur à
	<code>-ge</code>		supérieur ou égal à
	<code>-le</code>		inférieur ou égal à

Scripts portables – Comparaisons 2/4

Pour tester une chaîne de caractères, on procède comme ceci :

if test string1 = string2	égal à
!=	non égal à
>	supérieur à
<	inférieur à
if test -z string1	vérifie si la chaîne est vide
-n string1	vérifie si la chaîne est non vide

A chaque lettre correspond une valeur numérique (voir [man ascii](#)), cela permet d'utiliser les caractères < et >

Toujours mettre les chaînes de caractères entre « " » surtout les variables dont la valeur est une chaîne de caractère !

Scripts portables – Comparaisons 3/4

Rappel Dans les principes de Unix, TOUT est fichier.

Pour faire des tests sur des fichiers, on utilise fréquemment :

- **-e** fichier Vérifie si le fichier existe bien
- **-d** dossier Vérifie si le fichier est un dossier
- **-f** fichier Vérifie si le fichier est un fichier régulier
- **-L** fichier Vérifie si le fichier est un lien symbolique (**man ln**)
- **-r** fichier Vérifie si le fichier est accessible en lecture
- **-w** fichier Vérifie si le fichier est accessible en écriture
- **-x** fichier Vérifie si le fichier est exécutable
- **-s** fichier Vérifie si la taille du fichier est supérieur à zéro

Scripts portables – Comparaisons 4/4

Petit exemple récapitulatif :

```
#!/bin/bash
if [ $# -lt 1 -o $# -gt 2 ]
then
    echo "Usage : $0 argument1 [argument2]"
    exit 1
else
    if [ ! -e "$1" ] # verifie si le fichier $1 n'existe pas (!)
    then
        echo "le fichier $1 n'existe pas"
        exit 1
    elif [ -n "$2" -a -d "$2" ]
    then
        echo "$2 n'est pas vide et est un dossier"
    else
        echo "$1"
    fi
fi
exit 0
```

Scripts portables – Exercice 1

Écrire un script `exo5.sh` avec les consignes suivantes :

- Le script ne prend que 3 paramètres. En dessous ou au delà, afficher une erreur explicite et quitter le script
- Le premier paramètre doit être un nom de fichier. Le deuxième, le dossier dans lequel est contenu le fichier. Le troisième, un mot de votre choix
- Vérifier dans une seule condition que le fichier (Concaténer le deuxième et premier paramètre) existe ET qu'il n'est pas vide. Si une des conditions n'est pas remplie, afficher une erreur explicite et quitter le script
- Si la condition précédente est validée, chercher une occurrence du troisième paramètre dans ce fichier. Si la commande est un succès (ou pas). Afficher un texte de succès/échec à l'écran.

Scripts portables – le case

`case` correspond au `switch` du langage C

On peut effectuer un choix d'instructions selon la valeur d'une variable

`case` est souvent préféré à de multiples instructions `if...elif`

Exemple :

```
case $1 in
    1) echo "Vous avez saisie le choix 1"
        ;;
    2) echo "Vous avez saisie le choix 2"
        ;;
    yes) echo "Vous avez saisie yes"
        ;;
    no) echo "Vous avez saisie no"
        ;;
    *) echo "Choix inconnu !"
        ;;
esac
```

Scripts portables – Exercice 2

Écrire un script `exo6.sh` avec les consignes suivantes :

- Votre script affiche en premier lieu :

Quel est votre langage favori ?

- 1 - bash
- 2 - perl
- 3 - python
- 4 - c++
- 5 - Je ne sais pas

- Vous devez ensuite lire la réponse de l'utilisateur
- En fonction du choix numérique, afficher une réponse appropriée
- Si le choix de l'utilisateur n'est pas compris dans la liste, afficher un message d'erreur
- L'instruction `if` est interdite.

Scripts portables – le for 1/2

`for` permet l'utilisation de boucles itératives en bash

Les instructions entre `do` et `done` sont exécutées autant de fois qu'il y a de « mots » dans la liste après `in`

A chaque itération, on passe au mot suivant

S'il n'y a plus de mot, la boucle s'arrête

```
for variable-boucle [in liste-mots]
do
    liste-commandes-shell
    ...
done
```

Scripts portables – le for 2/2

Exemples :

```
for i in toto titi tutu
do
    echo $i
    cp $i $i.sav
done
```

```
for fichier in `ls /var/`
do
    echo $fichier
done
```

Scripts portables – Exercice 3

Écrire un script `script7.sh` avec les consignes suivantes :

- Utiliser une boucle `for` pour vérifier la présence des 20 premières machines de votre sous-réseau via la commande `ping`
- Enregistrer dans le fichier `/tmp/machine_ok` la liste des adresses IP des machines qui ont répondu.
- Enregistrer dans le fichier `/tmp/machine_ko` la liste des adresses IP des machines qui n'ont pas répondu.
- Afficher à la fin le nombre de machine qui ont répondu et le nombre de machine qui n'ont pas répondu
- Afficher la liste des adresses IP qui ont répondu

Astuces :

- Dans un autre terminal vous pouvez suivre le contenu des fichiers avec la commande `tail`
- Utiliser la commande `seq`

Scripts portables – le while 1/2

while permet d'exécuter une boucle basée sur le résultat d'un **test**

Tant que la condition exprimée par la liste derrière le mot clé **while** est vraie, le shell exécute le corps de la boucle et teste la condition :

```
while [ CONDITION ]  
do  
    liste-commandes-shell  
done
```

Exemple :

```
MIN=$1
```

```
while [ $MIN -gt 0 ]  
do  
    echo "plus que $MIN minutes" ; sleep 60  
    ((MIN=MIN-1))  
done  
  
echo fini
```

Scripts portables – le while 2/2

Autre exemple :

```
echo "Tapez le mot de passe :"  
read PASSWORD
```

```
SECRET=""  
while [ "$PASSWORD" != "$SECRET" ]  
do  
    if [ -z "$SECRET" ] ; then SECRET="$PASSWORD" ; fi  
    echo "Mot de passe incorrect."  
    echo "Tapez le mot de passe :"  
    read PASSWORD  
done
```

```
echo "Mot de passe correct."  
echo "Bienvenue sur les ordinateurs secrets du Pentagone."
```

Scripts portables – Exercice 4

Écrire un script `exo8.sh` avec les consignes suivantes :

- Faire une boucle qui tournera tant que le fichier `/tmp/existence` n'existe pas
- Lancer le script et le laisser tourner jusqu'à ce que le fichier soit créé
- Arrêter le script sans arrêter le processus

Astuces :

- Utiliser la commande `sleep` à chaque tour de boucle
- Utiliser 2 terminaux pour faire l'exercice

Scripts portables – les fonctions 1/4

Bash permet de définir des fonctions.

Elles sont utiles pour structurer et factoriser le code.

Elles se présentent de la manière suivante :

```
ma_fonction() # Egalement, function ma_fonction
{
    liste-de-commandes shell
}
```

Il est vivement conseillé de nommer les fonctions en minuscules.

Le caractère `_` est autorisé ainsi que les chiffres.

Une fonction n'est appelée qu'après sa déclaration. Exemple :

```
somme()
{
    echo "fonction somme $1 + $2 = $(( $1 + $2 ))"
}
somme 21 21
```

Scripts portables – les fonctions 2/4

Tout comme les scripts, elles peuvent prendre des arguments.

Dans ce cas, la fonction est appelée comme ceci :

```
ma_fonction argument1 argument2
```

Les arguments sont les mêmes qu'avec un script :

```
ma_fonction()  
{  
    echo $#  
    echo $*  
    echo $1 $2  
}
```


Scripts portables – les fonctions 3/4

Une fonction peut en appeler une autre à condition de respecter l'ordre des déclarations.

Exemple :

```
ma_fonction2()  
{  
    echo "je suis la fonction 2"  
}  
  
ma_fonction1()  
{  
    echo "je suis la fonction 1 et j'appelle la fonction 2"  
    ma_fonction2  
}  
ma_fonction1
```

Scripts portables – les fonctions 4/4

De même que les fonctions ont leurs propres paramètres, elles ont aussi leurs variables.

Toute variable déclarée dans une fonction est globale à tout le script. Il faut faire attention si une fonction écrase la valeur d'une variable du script.

Pour différencier les variables d'une fonction, on utilise le mot clé `local`

Exemple :

```
VAR="global variable"
function local_vs_global
{
    echo '2' $VAR
    local VAR="local variable"
    echo '3' $VAR
}
echo '1' $VAR
local_vs_global
echo '4' $VAR
```

Conseil de l'admin

Les fonctions accompagnées de commentaires permettent :

- Une meilleure lisibilité
 - Un programme sans fonction n'est lisible que s'il est petit (20 lignes)
- Une meilleure souplesse
 - Plus besoin de tout corriger, vos fonctions sont des briques encapsulées les unes sur les autres
- Un gain d'efficacité
 - Plus facile de cerner un bug dans des petits blocs que dans tout un tas de code

Sinon, pensez simplement à vos collègues qui devront relire vos scripts.

Scripts portables – L'import de fichiers

Le caractère `.` ou la commande `source` permettent d'importer un fichier bash dans un script tel que :

```
./chemin/vers/mon/fichier
```

C'est utile pour rapatrier des variables de configuration ou des fonctions shells

import.sh

```
USER="fanfan"
PASS="password"

authenticate()
{
    if [ "$USER" = "$1" -a
"$PASS" = "$2" ]
    then
        echo "Valide"
    else
        echo "Refuse"
    fi
}
```

test_import.sh

```
#!/bin/bash

. import.sh

echo -n "Votre login :"
read LOGIN
echo -n "Votre mot de passe :"
read MDP

authenticate $LOGIN $MDP

exit 0
```

Scripts portables – Exercice 5 1/2

Dans le script `fonctions.sh` :

- Écrire une fonction `error` qui prend au moins 1 paramètre et qui renvoi un affichage du type `"[ERROR] "` suivi des paramètres qui sont passés à la fonction.
- L'instruction `exit 1` à la fin de la fonction est obligatoire
- Écrire une fonction `backup`
- Elle demande 2 paramètres en mode interactif (`read`) :
 - le nom complet du fichier de sauvegarde (ex : `/home/user/backup`)
 - les noms complets des fichiers à sauvegarder séparés par des espaces (ex : `/etc/services /etc/hosts`)
- Dans une boucle vérifier que chacun des fichiers ou répertoires passés existent bien sinon envoyé une erreur en utilisant la fonction `error`
- Créer une archive compressée dont le nom de fichier de sauvegarde final doit ressembler à ceci : `backup_25-06-2015.tar.gz`

Scripts portables – Exercice 5 2/2

Dans le script `admin_tools.sh` :

- Importer le fichier `fonctions.sh` puis construire le script pour afficher les lignes suivantes (en mode interactif - `read`) :

```
$ ./admin_tools.sh
```

```
Choisissez votre outil :
```

```
1 – Sauvegarde
```

```
0 – Sortir du programme
```

```
Choix : 1
```

```
Backup :
```

```
Entrez le nom du fichier de backup : /home/formation/backup
```

```
Entrez les fichiers a sauvegarder separes par des espaces :
```

```
/etc/services /etc/hosts /etc/passwd
```

```
Le fichier sera cree dans /home/formation/backup_05-10-2012.tar.gz
```

```
# plusieurs lignes de retour de commandes
```

- Tant que l'utilisateur ne choisit pas un des choix proposés, afficher de nouveau le menu.
- Si la commande `tar` renvoie un mauvais code de retour, utiliser la fonction `error` pour signaler l'erreur
- Astuces : `man date`, `man tar`, relire la page qui explique `read`

Conseil de l'admin

Voici une petite liste de commandes que les admin doivent régulièrement utiliser :

<code>ps</code>	Affiche les processus du système	<code>lsuf</code>	Liste des fichiers ouverts
<code>free</code>	Affiche la mémoire disponible	<code>kill</code>	Détruire un processus
<code>top</code>	Idem que <code>ps</code> mais en temps réel (variantes à installer : <code>htop</code> , <code>top</code>)	<code>chmod</code>	Changer les droits de fichiers
<code>df</code>	Affiche l'espace disque disponible	<code>chown</code>	Changer le propriétaire de fichiers
<code>du</code>	Occupation disque d'un répertoire	<code>mount</code>	Monter des partages réseaux ou cd-rom
<code>vmstat</code>	Processus, mémoire et pagination	<code>history</code>	Affichage de l'historique de commande de l'utilisateur

Mécanismes avancés – les redirections 1/9

Imaginer : Dans un hachoir, vous mettez à son **entrée** de la viande. Puis, par un mécanisme du hachoir, une viande hachée arrive à sa **sortie**

Les processus Bash fonctionnent de même à la différence qu'ils utilisent **2 sorties**

Trois descripteurs de flux (canaux) sont disponibles pour tous les processus :

- **entrée** standard (numéro 0) (Clavier, par exemple)
- **sortie** standard (numéro 1) (Écran, par exemple)
- **sortie** d'erreur (numéro 2) (Écran, par exemple)

Potentiellement plus de canaux disponibles (intérêt théorique).

La bonne maîtrise d'un script Bash passe par la bonne manipulation des flux.

La redirection des flux permet de faire communiquer des commandes distinctes.

Exemple : recherche d'un terme dans le résultat d'un listing de fichier

Mécanismes avancés – les redirections 2/9

Le canal d'entrée standard

Symbole : < (ou 0<)

Utile pour envoyer des données à une commande.

Une commande qui attend une entrée au clavier peut recevoir des données placées dans un fichier.

Par exemple, on peut lire un fichier ainsi :

```
$ tail < monfichier
```

Cela équivaut en réalité à :

```
$ tail monfichier
```

Mécanismes avancés – les redirections 3/9

Les « here » scripts

Symbole : `<< EOF`

Permet de transmettre du contenu sur l'entrée standard jusqu'à la lecture du mot `EOF`

Pratique par exemple pour écrire plusieurs lignes dans un fichier.

Pratique pour lire plusieurs lignes depuis l'entrée standard.

Exemple avec la commande `cat` :

```
$ cat << !FINDEFICHIER  
Schtroumpf grognon  
toto  
!FINDEFICHIER
```

Affichera :

```
Schtroumpf grognon  
toto
```

Mécanismes avancés – les redirections 4/9

Canal de sortie standard

Symbole : `>` (ou `1>`)

Utile pour écrire la sortie d'une commande dans un fichier.

Peut être pratique pour éviter d'encombrer l'affichage du terminal (la sortie standard est par défaut dirigée sur l'écran !).

Par exemple, on peut sauvegarder les résultats d'une recherche :

```
$ grep tcp /etc/services > /tmp/fichier  
$ cat /tmp/fichier
```

On peut aussi écrire dans un fichier directement :

```
$ echo 'schtroumpf grognon' > liste_de_schtroumpfs
```

Mécanismes avancés – les redirections 5/9

Canal de sortie d'erreur

Symbole : 2>

Utile pour filtrer les erreurs (ou ne pas les afficher !)

Peut être pratique pour éviter d'encombrer l'affichage du terminal (la sortie d'erreur est par défaut aussi dirigée sur l'écran !).

Par exemple, on peut ne rien afficher si une commande échoue :

```
$ ls /etc/zzz 2> /dev/null
```

On peut aussi écrire dans un fichier directement :

```
$ cat /etc/zzz 2> liste_derreurs
```

Mécanismes avancés – les redirections 6/9

Rediriger la sortie d'erreur vers la sortie standard

Symbole : `2>&1`

Utile pour ne rien afficher en sortie d'une commande en redirigeant aussi la sortie standard vers `/dev/null` ! (pratique dans les scripts).

Très utilisé pour commandes en arrière-plan, qui peuvent afficher des sorties « n'importe quand ».

Par exemple, on peut laisser tourner une tâche courante en fond :

```
$ mise_a_jour_bdd > /dev/null 2>&1
```

Mécanismes avancés – les redirections 7/9

Concaténation à la fin d'un fichier

Symbole : >>

Par défaut les redirections vers des fichiers écrasent les contenus de ces derniers
Ce symbole permet d'ajouter à la fin du fichier le flux courant.

Par défaut, c'est la sortie standard qui est prise.

Par exemple, on peut construire une liste rapidement :

```
$ echo 'schtroumph grognon' >> liste_schtroumphs  
$ echo 'grand schtroumph' >> liste_schtroumphs
```

Mais aussi :

```
ls /etc/zzzz 2>> errors  
ls /etc/aaaa 2>> errors
```

Mécanismes avancés – les redirections 8/9

Les tubes (pipe)

Symbole : | (AltGr+6)

Permet de rediriger la sortie standard d'une commande vers l'entrée standard d'une autre commande.

Indispensable pour pouvoir enchaîner des actions.

Exemple : chercher un terme dans une liste de fichiers, dont les noms de fichiers correspondent aussi à un motif

```
$ find /var/log -name "*.log" | grep -i Error
```

Très couramment utilisé, c'est la base de la philosophie du shell que de pouvoir enchaîner des commandes !

Mécanismes avancés – les redirections 9/9



tee

La redirection d'un flux dans un fichier ne permet plus de l'utiliser pour une autre action (Exemple : affichage à l'écran).

tee permet de dupliquer l'entrée standard pour l'afficher également à l'écran.

Pratique pour pouvoir lire et écrire des logs en même temps

Exemple : sauvegarder les logs d'un script et les afficher en temps-réel à l'écran :

```
$ grep net /etc/* | tee resultats
```

L'option **-a** permet d'ajouter la sortie à la fin du fichier plutôt que de l'écraser :

```
$ grep net /etc/* | tee -a resultats
```


Mécanismes avancés – Exercice 1

Reprenez votre script `fonctions.sh` et rajouter une fonction `system_info` qui renvoie l'affichage approximatif (hors commentaires) suivant :

Hostname : bahamut # Rappelez vous l'environnement

CPU : 8 (Intel(R) Core(TM) i7-3610QM CPU @ 2.30GHz) # lscpu et /proc/cpuinfo

	total	used	free	shared	buffers	cached	
Mem:	7876	2418	5457	0	70	968	# <u>free, affichage en</u>
							<u>Mega Octets</u>

Uptime : 18:24:42 up 3:10, 0 users, load average: 0.03, 0.07, 0.08

Disk Space Usage						# <u>Affichage en Go</u>
Filesystem	Size	Used	Avail	Use%	Mounted on	
Rootfs	7.0G	5.3G	1.4G	80%	/	

Internet connection State: OK # ping

- Modifier `admin_tools.sh` pour prendre en compte la nouvelle fonction
- L'affichage de la commande `ping` ne doit pas s'afficher pendant l'exécution

Mécanismes avancés – Exercice 2

- A l'aide des mécanismes de Bash (pas d'éditeur de texte !), créer un fichier nommé `joueurs_tennis.txt` contenant la ligne suivante :
`nadal`
- En une commande, ajouter à la fin du fichier précédent le joueur `djokovic`
- Copier le fichier `/etc/services` vers le fichier `service.local` (utilisation de la commande `cp` interdite !)
- Exécuter `tail` sur le fichier `service.local`, rediriger les résultats dans `match.stdout` et les sorties d'erreurs dans `match.stderr`
- Même chose mais avec le fichier inexistant `service2.local`
- Calculer le hash MD5 du fichier `service.local`, l'afficher à l'écran ET l'écrire également dans le fichier `service.local.hash`

Astuce : utiliser la commande `md5sum` (RTFM !)

Mécanismes avancés – les opérations mathématiques 1/3

Le bash offre plusieurs solutions pour faire des calculs mathématiques.

Les calculs d'entier :

Usage de let

```
let ADDITION=4+5 # ou let ADDITION=$1+$2
```

typeset et/ou declare permettent de forcer le type d'une variable

```
typeset -i SOUSTRACTION
```

```
SOUSTRACTION=18-5
```

```
declare -i MULTIPLICATION
```

```
MULTIPLICATION=7*6
```

Le groupement de commandes

```
DIVISION=$((42 / 2))
```

```
MODULO=$((42 % 4))
```

Mécanismes avancés – les opérations mathématiques 2/3



Les calculs de nombres flottants se font de préférence avec l'outil **bc**

```
# Simple calculatrice en Bash
echo -n "Entrer 2 nombres : "
read NUM1 NUM2

echo "Resultat avec 2 chiffres apres la virgule"
echo "scale=2; $NUM1 / $NUM2" | bc -l

echo "10 chiffres apres la virgule"
echo "scale=10; $NUM1 / $NUM2" | bc -l

echo "Retour sans chiffres après la virgule"
echo "$NUM1 / $NUM2" | bc -l
```

Mécanismes avancés – les opérations mathématiques 3/3



Enfin, `bc` offre des outils de conversion de nombres.

On peut également préciser à `bc` la base d'entrée (`ibase`) et celle de sortie (`obase`) comme ceci

```
echo "ibase=10;obase=2;128" | bc
```

Mécanismes avancés – Meta characters 1/2

La liste des principaux « meta characters » (la suite):

- `|`, permet la communication par tubes de 2 commandes. Exemple : `ps
aux | grep ssh`
- `(...)`, regroupe des commandes. Exemple : `(echo "Liste :"; ls) >
liste.txt`
- `>`, redirige la sortie standard
- `>>`, redirige la sortie standard sans écrasement
- `2>`, redirige la sortie d'erreur
- `2>>`, redirige la sortie d'erreur sans écrasement
- `<`, redirige l'entrée standard
- `<<word`, redirige l'entrée standard jusqu'à écrire `word`

Mécanismes avancés – Meta characters 2/2

La liste des principaux « meta-characters » (la suite):

- `&&`, chaîne les commandes. Si la première commande ne renvoie pas d'erreur, la deuxième s'exécute. Exemple : `$ apt-get update && apt-get upgrade`
- `||`, chaîne les commandes. Si la première commande renvoie une erreur, la deuxième s'exécute. Exemple : `$ cd /etc/toto || echo "KO"`
- `[...]`, tout ce qui est entre les crochets permet de matcher certains caractères. Exemples :

```
$ cat /etc/services | grep [0-9] # Affiche les lignes avec  
caracteres numeriques
```

```
$ cat /etc/services | grep [a-zA-Z] # Affiche les lignes avec  
des chaines de caracteres majuscules ou minuscules
```

Mécanismes avancés – ANSI-C

Dans Bash, certains caractères obtiennent une signification différente dès qu'on les fait précéder d'un '\':

- \a, fait sonner une alerte par le shell
- \b, équivalent à la touche clavier backspace. Exemple : `$ echo -e "coucou\b tout le monde"`
- \n, affiche un saut de ligne. Exemple : `$ echo -e "coucou\n"`
- \r, fait un retour chariot (déplacement du curseur au début de la ligne)
- \t, tabulation horizontale (touche Tab du clavier)
- \v, tabulation verticale. Exemple : `$ echo -e "coucou\v tout le monde"`

Mécanismes avancés – getopt

Un script shell un peu perfectionné pourra avoir de nombreux paramètres ou options :

```
$ program.sh -u toto -p hello -h -m
```

Le problème étant qu'il devient vite compliqué de devoir analyser tous les paramètres et options qu'on lui passe. Surtout si on se trompe d'ordre avec les paramètres.

On utilise une fonction intégrée de Bash : `getopts`. Exemple :

: apres une lettre signifie qu'on attend des options apres la lettre

```
while getopts u:p:hm OPTION
```

```
do
  case $OPTION in
    u)
      echo "option user : $OPTARG";;
    p)
      echo "option password : $OPTARG" ;;
    h)
      echo "option help : Usage ..." ;;
    m)
      echo "option Meuh : Meuh Meuh" ;;
  esac
done
```

Mécanismes avancés – Exercice 3

Écrire un script `calculatrice.sh` tel que :

- Le script puisse prendre 1 paramètre obligatoire et 2 paramètres optionnels.
- Le paramètre obligatoire (`-c`) prend en argument un calcul ou un nombre.
Exemple : `-c 42/6`
- `-i` : paramètre optionnel précisant la base d'entrée
- `-o` : paramètre optionnel précisant la base de sortie
- Par défaut, si l'un ou l'autre ou les 2 paramètres optionnels ne sont pas renseignés les bases d'entrée et de sortie sont des bases 10 (autres bases connues : 2, 8, 16 et 64)
- Exemples :

```
$ ./calculatrice.sh -c 50%4
```

```
2
```

```
$ ./calculatrice.sh -c 50-4 -i 10 -o 16
```

```
2E
```

```
$ ./calculatrice.sh -c 50*4 -o 2
```

```
11001000
```

```
$ ./calculatrice.sh -c "(-(50*4)+(42*-13))"
```

```
-746
```

Mécanismes avancés – Les tableaux 1/3

Les versions récentes de Bash supportent l'utilisation de tableaux mono dimensionnels.

Un tableau peut stocker un ensemble de variables sous la forme :

- clé(entier) => valeur

Exemple :

```
# Declaration d'un tableau
declare -a ARRAY
```

```
let count=0
while read LINE; do # Appuyer sur Ctrl+d pour arreter la boucle
    ARRAY[$count]=$LINE
    ((count++))
done
```

```
echo Nombre d elements: ${#ARRAY[*]}
echo Contenu du tableau ${ARRAY[*]}
echo Contenu du premier element ${ARRAY[0]}
```

Mécanismes avancés – Les tableaux 2/3

Un tableau peut s'initialiser de différentes manières :

```
$ ARRAY=(coucou hello "bonjour à tous")
```

Vous pouvez aussi affecter directement une valeur à une case choisie au hasard :

```
ARRAY[42]="reponse a toutes les questions de la vie, l'univers et le  
reste"
```

Si vous utilisez ce moyen sur un tableau qui n'existe pas, cela le crée également.

La dernière façon permet de transformer la façon dont `read` stocke ce qu'il lit :

```
$ read -a ARRAY  
Bonjour tout le monde
```

```
$ echo ${ARRAY[0]}  
Bonjour
```

Mécanismes avancés – Les tableaux 3/3

Afficher la taille d'un élément

```
$ echo ${#ARRAY[0]}
```

Ajouter un élément à la dernière case du tableau

```
$ ARRAY[${#ARRAY[*]}]=nouvel_element
```

Ajouter un élément à la première case du tableau

```
$ ARRAY=( nouvel_element ${ARRAY[*]} )
```

Détruire un indice du tableau

```
$ unset ARRAY[2]
```

Détruire un tableau

```
$ unset ARRAY
```

Mécanismes avancés – Le select

L'instruction `select` permet d'afficher un menu interactif à partir de la variable globale `PS3`

Plus besoin de `echo`, `read` ou encore `case` pour ce genre de menu :

```
PS3='Choisissez un chiffre: '
```

```
select WORD in "linux" "bash" "scripting" "tutorial"
do
    echo "Le mot choisi est: $WORD"
# Break, permet d'éviter les boucles infinies
    break
done
```

Mécanismes avancés – Exercice 4

Modifiez simplement votre fichier `admin_tool.sh` pour utiliser la commande `select` afin d'afficher votre menu interactif

Mécanismes avancés – Les signaux 1/2

Lorsqu'un programme est exécuté, il devient un processus.

Le processus est soumis à une liste de signaux que d'autres programmes ou l'utilisateur peuvent envoyer

Les signaux les plus courants sont :

- **INT** (n°2), signal d'interruption. Généré par **Ctrl+c**. Le processus qui reçoit ce signal s'arrête et libère le prompt
- **QUIT** (n°3), Core dump : génère un fichier core. Généré par **Ctrl+d**.
- **KILL** (n°9), signal d'arrêt immédiat (à utiliser en dernier recours)
- **TERM** (n°15), envoie un signal au programme pour le terminer
- **STOP** (n°24), signal demandant au programme de se suspendre (**Ctrl+z**)

La liste totale des signaux peut être obtenue en tapant la commande **kill -l**

Mécanismes avancés – Les signaux 2/2

La commande `kill` permet d'envoyer des signaux aux processus actifs. Exemple :

```
$ kill -s 24 1042
```

1042 correspond au numéro du processus (PID) de votre programme. Le numéro est récupérable, entre autres grâce à la commande `ps aux | grep nom_du_programme`

La commande `trap` permet de capturer les signaux envoyés à un processus et d'inhiber ou transformer le comportement du signal. Exemple :

```
$ trap 'rm /tmp/toto ; exit 1' 2 STOP
```

La ligne ci-dessus va capturer `Ctrl+c` et `Ctrl+z` pour exécuter une commande personnalisée à la place (premier paramètre)

Dans un script Bash, le premier paramètre de `trap` peut aussi être une fonction. De même, il est préférable de déclarer les `trap` au début du script

Les deuxième, troisième, etc paramètres sont une liste de signaux séparés par des espaces nommés avec leur nombre ou leur valeur (voir `kill -l`)



Questions ?